

Axon Key Lifecycle and Bootstrap

Supporting signature management for the Axon invocation protocol

Silan Hu

National University of Singapore

v1, April 21, 2026

Abstract

This document specifies key lifecycle, bootstrap, and signature management mechanisms for Axon-based systems. It defines how cryptographic keys are created, registered, rotated, and revoked, and how trust roots are established between Axon runtimes and their clients. It does *not* define invocation envelope semantics: the structure of invocations, their canonical bytes, and the binding of signatures to invocation fields are specified in the Axon invocation protocol (RFC 001) and its underlying axiom document. This document is complementary to those specifications and defines how keys are managed, not what is signed.

Contents

1	Scope	2
2	Adversary model	2
3	Identity model	3
3.1	Key record	3
4	Subsumption of the legacy (node, principal) dual-binding model	4
5	Bootstrap: separating authorisation from identity	5
5.1	The problem bootstrap solves	5
5.2	Bootstrap tokens	5
5.3	Durable keypair registration	6
5.4	Bootstrap threats addressed	6
6	Key lifecycle	6
6.1	States	6
6.2	Rotation	7
6.3	Revocation	7
6.4	Recovery	7
7	Trust roots and host verification	7
8	Signature suite	8
9	Protocol surface	8
10	Relationship to Axon invocation protocol	9
11	Open extensions	9

12 Conclusion

10

1 Scope

[axiom] This document’s scope is limited to:

- Identity-key registration (node, agent, trust-root).
- Bootstrap tokens and the separation between bootstrap authorization and durable identity.
- Key rotation, revocation, and recovery procedures.
- Trust-root establishment and runtime verification (analogous to SSH `known_hosts`).
- Cryptographic algorithm agility and signing-suite requirements.

[axiom] This document does *not* specify:

- The structure of an Axon invocation. *See RFC 001 §3.*
- The canonical byte layout used as input to signatures. *See RFC 001 §4.*
- Which fields are covered by the caller’s signature versus the callee’s signature. *See AXIOM.tex §5.3 and RFC 001 §5.*
- Ability naming, versioning, or snapshots. *See AXIOM.tex §5.7 and RFC 001 §3.6.*

Note

Relationship to prior artefacts. An earlier document, `invocation_security_v1.tex`, conflated envelope definition with key management and defined a nine-field invocation tuple that predated the seven-parameter axiom. That document has been withdrawn. The envelope definition now lives in Axon’s RFC 001 as the sole authority; the key management content has been narrowed and carried forward into this document under the present scope.

[axiom] This document is *complementary* to RFC 001 and AXIOM.tex. Conformance to either of those documents without conformance to this one yields an invocation protocol whose signatures cannot be registered, rotated, or revoked in an interoperable manner. Conformance to this document without the other two yields a key management system with nothing to sign.

2 Adversary model

[derived] The adversary $\mathcal{A}$ is computationally bounded and:

- can observe, delay, reorder, replay, and drop any message on the network (Dolev-Yao-like transport);
- controls some but not all endpoints and may hold compromised keys;
- cannot forge signatures over canonical bytes without possessing the corresponding private key;
- cannot break the cryptographic hash used in canonicalisation (SHA-256 or successor);

- does not control the trust-root verification channel during initial pinning (out-of-band authenticated).

[**derived**] Host compromise after key registration is in scope for key rotation and revocation; host compromise before key registration is out of scope and mitigated only by the trust-root pinning discipline of §7.

3 Identity model

[**axiom**] The specification recognises three layers of cryptographic identity:

- **Agent identity** (K_A). A keypair bound to an Axon agent (human user, service account, autonomous agent). K_A is the key that signs invocation envelopes on the agent's behalf; its URI and profile, per RFC 001 §3.1, name the agent in the envelope's **caller** or **callee** field.
- **Node identity** (K_N). A keypair bound to an Axon runtime instance (host, process, container). K_N is not used to sign invocations in the current Axon model — the envelope carries only agent signatures — but is used to sign runtime-to-runtime control-plane messages (handshake, agent-card exchange, trust-root statements) and to attest the node's identity during bootstrap.
- **Trust root identity** (K_R). The keypair or certificate authority that endorses the runtime's public identity, permitting clients to verify the runtime itself. This is the runtime's counterpart to SSH's `known_hosts` or a TLS root certificate.

Note

Relationship to the previous (node, principal) model. Earlier drafts of this material modelled invocation security as a two-axis binding over (*node origin*, *principal authorisation*). That model has been superseded by the Axon axiom's (*caller*, *callee*) dual-signature structure (AXIOM.tex §5.3): the caller signs the invocation's canonical bytes, the callee signs the receipt's canonical bytes, and the two signatures together establish cross-deployment non-repudiation. The earlier node-signature role survives only at the control-plane layer defined here (runtime handshake, trust-root attestation); it is not an envelope signature. Formal derivation is given in §4.

[**axiom**] All identities use the same underlying signature suite (§8) so that evidence is uniform across the protocol and pipelines are algorithm-agile.

3.1 Key record

[**axiom**] Every identity is associated with a *key record* of the following shape:

```
KeyRecord {
  subject:    { kind: Agent | Node | Root, uri: string, profile: string },
  algorithm:  Ed25519 | ECDSA-P256 | ... (see $sign-suite),
  version:    u32,                               // monotonically increasing
  public_key: bytes,
  state:      Active | Rotated | Revoked,
  mode:       Strong | Weak,                      // agents only; nodes are always Strong
  created_at: int64,
  rotated_at: int64,
  expires_at: int64,                             // 0 = no expiry
  fingerprint: bytes                           // SHA-256 of public_key
}
```

```
}

```

[derived] The `subject.uri` and `subject.profile` fields are the URA URI and profile selector specified in RFC 001 §3.1; this document does not re-define them. A registry implementing this spec resolves an invocation’s `caller.uri` or `callee.uri` to the appropriate `KeyRecord` via this field.

[derived] A *strong agent identity* is one whose private component is generated and held exclusively within the agent’s own trust boundary (e.g. a user’s browser via WebCrypto [5], a developer’s local machine, or a hardware token). A *weak agent identity* is one whose private component is held by an intermediary signing on the agent’s behalf (e.g. a backend service-account key signing “as user U ”). Weak identity is marked in the key record and surfaced in the receipt’s audit record. Both kinds are valid Axon agents under RFC 001; the distinction is evidentiary strength, not kind.

4 Subsumption of the legacy (node, principal) dual-binding model

This section records, as a formal artefact, the theorem that justifies replacing the earlier (*node, principal*) dual-binding model with the Axon (*caller, callee*) dual-signature model.

Definition 1 (Caller signature). Let I be an Axon invocation and $C(I)$ its canonical bytes per RFC 001 §4.1. A *caller signature* over I is $\sigma_c = \text{Sign}(sk_c, C(I))$, where sk_c is the private key whose public counterpart is the key record registered to the URI named in $I.\text{caller}$.

Definition 2 (Callee signature). Let R be a terminal receipt for the invocation I and $C(R)$ its canonical bytes per RFC 001 §4.3. A *callee signature* over R is $\sigma_e = \text{Sign}(sk_e, C(R))$, where sk_e is the private key whose public counterpart is the key record registered to the URI named in $R.\text{callee_binding}$.

Theorem 1 (Dual-signature accountability). *Given a triple $(I, R, (\sigma_c, \sigma_e))$ such that σ_c verifies under Definition 1 and σ_e verifies under Definition 2, and such that R binds back to I by carrying I ’s nonce and causal context verbatim in its canonical bytes (RFC 001 §4.3), then:*

1. **Invocation non-repudiation.** *The principal holding sk_c cannot later deny authoring I with the specific caller, callee, ability, subject, nonce, causal_context, args the canonical bytes carry.*
2. **Execution attestation.** *The principal holding sk_e cannot later deny executing the invocation named by R to the specific terminal state and ability snapshot the receipt’s canonical bytes carry.*
3. **Cross-deployment accountability.** *A third party holding only the public counterparts of sk_c and sk_e , and the receipt bytes, can confirm both propositions without any trust in either principal’s runtime.*

Proof sketch. (1) follows from the unforgeability of the signature scheme over $C(I)$, together with the fact that $C(I)$ is a complete deterministic encoding of the seven-tuple (AXIOM.tex §2.2.1 minimality) — the principal cannot claim a different I was signed. (2) follows analogously for $C(R)$, and additionally from the binding of R to I via the nonce and causal context, which are in $C(R)$: R cannot be re-attributed to a different invocation without producing a different $C(R)$ and thus a failed signature. (3) is the immediate consequence: verification requires only the public keys and the two signed byte sequences. $\square$

Theorem 2 (Subsumption of (node, principal)). *The legacy (node, principal) dual-binding model, in which σ_N attests node origin and σ_P attests principal authorisation, is subsumed by the (caller, callee) dual-signature model in the following sense: any property expressible in the*

legacy model is expressible in the Axon model, and Axon’s causal-context parameter additionally expresses delegation, capability grants, and multi-predecessor causation that the legacy model could not.

Proof sketch. A legacy node-origin claim (N transported I) is expressible in the Axon model by having the runtime at N participate as an agent in its own right: either (a) the caller is itself hosted at N , in which case σ_c attests both authorship and origin; or (b) the caller is a distinct agent and N ’s participation is recorded as a runtime event (for example, a signed handshake receipt cited in $I.\text{causal_context}$), at which point the node-origin fact becomes a causal predecessor covered by σ_c rather than a separate envelope signature.

A legacy multi-principal authorisation (k principals each signing the same $C(I)$) is expressible as k separate grant receipts, each authored by one principal, cited in $I.\text{causal_context}$ as a list. The Axon model requires exactly one caller signature on the envelope (INV-1 seven-tuple closure, RFC 001), so multi-principal authorisation is always indirect via grants — a strictly more general construction than simultaneous signing.

Conversely, the Axon model expresses forms the legacy model cannot: a DAG-shaped causal context (RFC 001 §3.3 list and merkle forms) has no legacy counterpart; a caller-signed ability pin (AXIOM.tex §5.7) has no legacy counterpart; a callee-signed receipt over execution outcome has no legacy counterpart (the legacy model had only invocation-side signatures, no execution-side). Hence the Axon model is strictly stronger. $\square$

Invariant

(Binding non-substitutability.) A node signature σ_N , valid at the runtime handshake or control-plane layer, does not substitute for a caller signature σ_c on an invocation. A caller signature does not substitute for a callee signature on a receipt. An invocation missing σ_c or a receipt missing σ_e is not valid, regardless of what other signatures are present.

5 Bootstrap: separating authorisation from identity

5.1 The problem bootstrap solves

[axiom] Identity registration is itself an operation that must be authorised, yet the identity being registered does not yet exist at authorisation time. This chicken-and-egg problem is solved in OpenSSH [2] by separating two roles: a short-lived *bootstrap authorisation* (e.g. the password accepted by `ssh-copy-id`), which authorises exactly one transaction — the placement of a public key into `authorized_keys` — and a *durable identity* (the keypair) that authenticates every subsequent session. Axon adopts the same separation.

5.2 Bootstrap tokens

[axiom] A *bootstrap token* is a high-entropy, server-issued, single-use credential. It satisfies:

- **High entropy.** At least 256 bits of cryptographic randomness, rendered in a URL-safe encoding.
- **Single-use.** The server atomically consumes it on validation; a second presentation fails.
- **Short-lived.** Validity is bounded by an explicit expiry (recommended: ≤ 15 minutes from issuance).

- **Narrowly scoped.** Authorises one transaction of one kind: registration of a single keypair under a single named identity.

[axiom] The bootstrap token authorises the transaction “register public key K as the durable identity of subject S within profile-determined naming scope.” That is its entire authority. It is not an identity and is not accepted as evidence of any proposition about an Axon invocation or receipt after the registration transaction completes.

5.3 Durable keypair registration

[axiom] Durable keypair registration proceeds as follows. Let the prospective subject be S , the subject’s naming scope be fixed by the profile, and the bootstrap token be τ :

1. S locally generates a fresh keypair (sk_S, K_S) using the operating system’s cryptographic RNG (§8).
2. S computes a *possession proof* $\pi = \text{Sign}(sk_S, C_{\text{reg}})$ where C_{reg} is the canonical encoding of the registration intent: $(\tau, K_S, S_{\text{claim}}, t_{\text{now}})$.
3. S sends $(\tau, K_S, S_{\text{claim}}, t_{\text{now}}, \pi)$ to the server’s **RegisterKey** endpoint over an authenticated channel (TLS 1.3 or stronger).
4. The server atomically: (a) validates τ and marks it consumed, (b) verifies π under K_S , (c) writes a **KeyRecord** with the new key and state **Active**, (d) returns a confirmation with the key’s **fingerprint** and **version**.

[derived] The possession proof π serves three roles: it binds the registration intent to K_S ’s private component, it defends against an adversary substituting K_S for an attacker-chosen key during transport, and it provides a first exercise of the signing suite that ensures K_S is well-formed.

5.4 Bootstrap threats addressed

[derived] The bootstrap procedure defends against:

- **Token exfiltration.** Mitigated by short expiry and single-use atomicity.
- **Key substitution in transit.** Mitigated by the possession proof π .
- **Replay of the registration request.** Mitigated by the atomic token consumption and the timestamp t_{now} .
- **Bootstrap-token as durable identity.** Mitigated by the axiom that τ authorises exactly one transaction; no post-registration verifier accepts τ as evidence.

6 Key lifecycle

6.1 States

[axiom] A **KeyRecord** occupies exactly one of three states:

- **Active.** The key is accepted as evidence for current signatures.
- **Rotated.** The key has been succeeded by a newer version; it is accepted as evidence for signatures produced during its validity window but not for new signatures.

- **Revoked.** The key is rejected as evidence for all signatures, regardless of production time. Revocation is monotonic and cannot be undone.

[axiom] The revocation relation is monotonic: no key transitions from **Revoked** back to **Active** or **Rotated**. A revoked key’s holder must bootstrap a new key via the procedure of §5.

6.2 Rotation

[axiom] *Rotation* replaces an **Active** key K_{old} with a new key K_{new} of the same subject, such that:

- K_{new} is freshly generated and registered via a **RotateKey** request, which is itself signed by sk_{old} (i.e. rotation is authorised by the current key).
- K_{old} transitions to **Rotated** atomically with the activation of K_{new} ; no window exists in which both are **Active**.
- The **KeyRecord**’s **version** increments.

[derived] A signature produced with sk_{old} at time t verifies if and only if K_{old} was in **Active** or **Rotated** state at time t , and the current state is not **Revoked**. This is the standard “rotate forward but honour past” semantics.

6.3 Revocation

[axiom] *Revocation* is authorised by one of:

- the key-holder, via a **RevokeKey** request signed by sk (self-revocation);
- the trust root K_R , via a **RevokeKey** request signed by sk_R (emergency revocation by the runtime’s operator, for key compromise or personnel change);
- an automatic policy engine, e.g. on expiry, signed by the runtime’s internal service key (which itself has a registered **KeyRecord**).

[axiom] On revocation, any invocation or receipt produced with the revoked key after the revocation’s timestamp is rejected by admission. The question of whether signatures produced *before* the revocation timestamp remain valid is a deployment policy decision, not a protocol invariant: some deployments treat revocation as retroactive (all signatures invalidated), others treat it as forward-only (past signatures remain valid for audit but the key cannot sign new invocations).

6.4 Recovery

[deferred] Key-loss recovery (a subject has lost access to sk but wishes to preserve their identity S) is not specified in v1. Three candidate mechanisms are noted for future work: (a) pre-registered recovery keys, (b) trust-root-mediated re-issuance under operator attestation, (c) social recovery via k -of- n peer signatures. The default v1 posture is that a lost key requires a new identity: S cannot be recovered, only reconstituted as S' .

7 Trust roots and host verification

[axiom] Every Axon runtime is identified by a *host key* K_{host} endorsed by a *trust root* K_R . A client verifies the runtime by:

1. fetching the runtime’s endorsement certificate, which contains K_{host} signed by K_R ;

2. verifying the endorsement signature against the client’s pinned copy of K_R ;
3. confirming that the TLS session’s server key matches K_{host} (i.e. standard TLS key-pinning);
4. rejecting the session if any step fails.

[axiom] The initial pinning of K_R is an out-of-band trust establishment, analogous to accepting an SSH host key on first connection (`known_hosts`). Two modes are supported:

- **Trust on first use (TOFU).** The client accepts the trust root on first contact and pins it locally. Subsequent sessions verify against the pin.
- **Attested pinning.** The client receives the trust root via a separate authenticated channel (deployment manifest, signed announcement, hardware trust anchor) and pins it at that point.

[derived] TOFU is appropriate for developer environments and for small-scale deployments; attested pinning is required for any deployment that claims cross-organisation non-repudiation, because TOFU pinning leaves the initial contact vulnerable to man-in-the-middle substitution.

8 Signature suite

[axiom] Implementations **MUST** support at least:

- **Ed25519** (RFC 8032 [1]) for the default signing algorithm.
- **ECDSA-P256-SHA-256** (FIPS 186-5 [4]) for interoperability with ecosystems requiring NIST curves.
- **SHA-256** for all hash operations, including canonicalisation input to RFC 001’s signing regions.

[derived] Post-quantum agility is not required for v1, but the `KeyRecord.algorithm` field is a forward-compatible enum; a future version adding FIPS 204 (ML-DSA) or FIPS 205 (SLH-DSA) need not break existing deployments. Implementations **SHOULD** plan for dual-signing windows when migrating.

[axiom] Implementations **MUST** reject key records whose `algorithm` is not in the above list, or in a profile-specified extension of it. A runtime seeing an unknown algorithm fails closed: it does not verify the signature and rejects the invocation or receipt.

9 Protocol surface

[axiom] This specification requires the following wire endpoints, each with signed request/response shapes defined in Appendix A (deferred to implementation spec):

- **IssueBootstrapToken** — operator-authorised, produces τ .
- **RegisterKey** — consumes τ , registers K_S .
- **RotateKey** — authorised by sk_{old} .
- **RevokeKey** — authorised by sk , sk_R , or policy engine.
- **LookupKey** — by URI, returns current `KeyRecord`.
- **ListKeyVersions** — by URI, returns historical records for audit.

- **GetTrustRoot** — returns the runtime’s endorsement certificate.

[derived] These endpoints are the control-plane counterpart of Axon’s data-plane invocation endpoints (RFC 001 §5). They are deliberately separated: an invocation never modifies key state, and a key-management request never carries an invocation envelope.

10 Relationship to Axon invocation protocol

The following table summarises, for the reader coming from either Axon or this document, how responsibilities are divided.

Concern	Defined in	Role of this document
Invocation envelope shape	RFC 001 §3	n/a
Canonical bytes of invocation	RFC 001 §4.1	n/a
Canonical bytes of receipt	RFC 001 §4.3	n/a
What the caller signature covers	RFC 001 §5, AXIOM.tex §5.3	Defines the key used to produce it
What the callee signature covers	RFC 001 §5, AXIOM.tex §5.3	Defines the key used to produce it
Agent identity URI + profile	RFC 001 §3.1, URA v2	Maps URI to <code>KeyRecord</code>
Key creation / registration	—	This document
Key rotation / revocation	—	This document
Trust root establishment	—	This document
Bootstrap tokens	—	This document
Algorithm agility	—	This document
Ability snapshot verification	AXIOM.tex §5.7, RFC 001 §3.6	n/a
Causal context encoding	RFC 001 §3.3, §4.2	n/a
Nonce dedup window	AXIOM.tex §5.1 I1, RFC 001 §5.2	n/a

11 Open extensions

[deferred] The following are out of scope for v1 and are listed to mark the boundary, not to commit to solutions:

- Post-quantum signature algorithms (ML-DSA, SLH-DSA).
- Social recovery and k -of- n threshold keys.
- Hardware attestation for key generation (TPM, Secure Enclave, HSM attestation).
- Federation between multiple trust roots and inter-runtime trust bridging.
- Formal Tamarin proofs of the dual-signature accountability theorem (Theorem 1).

12 Conclusion

This document specifies the key lifecycle, bootstrap, and signature management machinery that the Axon invocation protocol (RFC 001) depends on but does not itself define. The two documents together form a complete specification of how an Axon envelope becomes a cryptographically accountable object; either document alone is insufficient.

The key structural move in this revision is the abandonment of the legacy (*node*, *principal*) envelope-signature model in favour of Axon's (*caller*, *callee*) envelope-plus-receipt signature model. Theorem 2 records that this move is strictly a generalisation: every legacy binding is expressible, and Axon additionally expresses delegation, capability grants, DAG-shaped causal context, and ability-snapshot pinning, which the legacy model could not.

References

- [1] S. Josefsson and I. Liusvaara. *RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. IETF, 2017.
- [2] T. Ylonen and C. Lonvick. *RFC 4252: The Secure Shell (SSH) Authentication Protocol*. IETF, 2006.
- [3] E. Rescorla. *RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3*. IETF, 2018.
- [4] National Institute of Standards and Technology. *FIPS 186-5: Digital Signature Standard*. 2023.
- [5] World Wide Web Consortium. *Web Cryptography API*. W3C Recommendation, 2017.